Rebuild plan: Polymarket US bot

Goal and honest expectation

You want a bot that earns money on about $70, running on an AWS server. The venue is **Polymarket US**. It's the CFTC-regulated exchange, so you can legally trade it as a US resident, and it allows bots through its API (`MANUAL_ORDER_INDICATOR_AUTOMATIC`).

What the research measured on Polymarket US today:

- **Markets.** 91% of active markets are sports.
- **Spreads.** Active books are deep and tight. The median spread is 0.1 cents, with more than 25k contracts within 2 cents. Retail can't win a spread fight there.
- **Reward pools.** Liquidity reward pools are small: $5 to $20 per window, with a target size of 5,000 contracts. Payouts under $1.00 aren't paid out.
- **Maker rebate.** It's `0.0125 * C * p * (1-p)`, about $0.003 per contract at p=0.50. Taker fee is `0.0695 * C * p * (1-p)`.
- **Baskets.** In 6 of 20 mutually exclusive events, the sum of best bids was above $1.00 (up to 1.018). Taker fees on every leg wipe that out.

A $70 account can rest about 140 contracts at p=0.50. Against a 5,000-contract target, that's roughly 3% of one pool, or about $0.35 per window (inferred, not measured). **So expect cents to low dollars a day, not a living.** That's why this plan builds a **shadow mode first**. It measures what the strategy would have earned on live data, and real money only goes in if those numbers are positive.

> [!IMPORTANT] These are yours to do before live trading. The bot can't do them. 1. Create a Polymarket US account in the iOS app and pass the ID check. Florida isn't on any excluded list in the docs, but I haven't confirmed it, so check in the app. 2. Create an API key at polymarket.us/developer. You get a Key ID and a Secret Key, and the secret is shown only once. 3. Fund the account. Per the docs, depositing at least $10 or $50 earns a promotional trading credit.

Scope

- **New package:** `d:\neststock\scripts\polymarket_bot\pmus\`.
- **Old bot left in place:** `paper_trader.py` and the rest of the old Polymarket bot aren't touched in this phase. Deleting them is a follow-up once the new bot runs.
- **No routing around restrictions:** no VPN or geoblock-workaround logic of any kind.

Data shapes

All in `pmus/models.py`, as frozen dataclasses. External JSON is parsed only in `venue.py`.

```
Side = Literal["BUY_LONG", "SELL_LONG", "BUY_SHORT", "SELL_SHORT"]
```

```
@dataclass(frozen=True)
```

```
class Market:
    slug: str; event_id: str | None; category: str
    tick: Decimal; min_qty: Decimal; active: bool; mutually_exclusive: bool

@dataclass(frozen=True)
class Level: price: Decimal; qty: Decimal

@dataclass(frozen=True)
class Book:
    slug: str; bids: tuple[Level, ...]; asks: tuple[Level, ...]; ts_ms: int   # bids desc, asks asc

@dataclass(frozen=True)
class IncentiveWindow:
    slug: str; program_id: str; pool_usd: Decimal; discount: Decimal
    target_size: Decimal; max_spread: Decimal | None; start: datetime; end: datetime; status: str

@dataclass(frozen=True)
class Quote:            # desired resting order, always post-only
    slug: str; side: Side; price: Decimal; qty: Decimal

@dataclass(frozen=True)
class LiveOrder:
    order_id: str; slug: str; side: Side; price: Decimal; qty_open: Decimal

@dataclass(frozen=True)
class Position: slug: str; net_qty: Decimal; avg_price: Decimal   # +long YES, -short YES

@dataclass(frozen=True)
class BasketOpp:
    event_id: str; direction: Literal["SHORT_ALL", "BUY_ALL"]
    legs: tuple[tuple[str, Decimal], ...]; gross_edge: Decimal; fees: Decimal; net_edge: Decimal
```

Money and prices use `Decimal`, never float. Every boundary rounds to the market's tick or to cents.

Module layout and responsibilities

File	Responsibility
`pmus/config.py`	A `Settings` dataclass read from env: `PMUS_KEY_ID`, `PMUS_SECRET_KEY`, `PMUS_MODE=shadow|live`, and the risk caps. Defaults to `shadow`.
`pmus/venue.py`	The only module that talks to the network. It covers public gateway reads (`https://gateway.polymarket.us`: `/v1/markets`, `/v1/events`, `/v1/markets/{slug}/book`, `/bbo`, `/v1/incentives`) and authenticated calls through the official `polymarket-us` SDK (`PolymarketUS(key_id, secret_key)`: `orders.create`, batched create and cancel up to 20, `orders.cancel_all`, `orders.list`, `portfolio.positions`, `account.balances`, plus the markets and

File	Responsibility
	private websockets). It parses everything into `models`. Timeouts on every call, and it backs off on 429s (limit is 25 req/s).
`pmus/fees.py`	`taker_fee(c, p)` and `maker_rebate(c, p)`, with banker's rounding to the cent, per docs.polymarket.us/fees.md.
`pmus/strategy/lip_quoter.py`	A pure function: `desired_quotes(market, book, window, position, budget) -> list[Quote]`. It joins the best price on one or both sides (discount^0), never improves through the spread by more than one tick, and never crosses. It skips the market if the book fails `max_spread` or the window isn't active. Size is the market's share of the budget divided by collateral per contract: `p` for a long, `1-p` for a short. Inventory skew means that once net position exceeds the per-market cap, it quotes only the side that reduces it.
`pmus/strategy/basket_scanner.py`	A pure function: `scan(events, books) -> list[BasketOpp]`. It computes SHORT_ALL (sum of bids − 1) and BUY_ALL (1 − sum of asks), each net of per-leg taker fees, and keeps only those with `net_edge >= min_edge`. **Phase 1 logs only.**
`pmus/engine.py`	The reconcile loop. Each tick it computes desired quotes and diffs them against live orders, giving cancel / create / keep sets. It submits through batched endpoints. Each market has an explicit state: `IDLE -> QUOTING -> PAUSED`, plus `FLATTENING` when caps are hit. Book updates come from the markets websocket, with a REST poll fallback every 5s.
`pmus/risk.py`	Hard gates checked before every submit: max total collateral in use, max per market, max open orders, a daily realized-loss stop, a kill-switch file (`STOP`), and a maintenance pause. It cancels everything at 05:55 ET Thursday and resumes at 08:05 ET. A gate breach moves the market to `FLATTENING` or `PAUSED`.
`pmus/shadow.py`	A paper fill model on live public data. A resting shadow quote fills when trades print through its price, after the estimated queue ahead of it. Each second it estimates the LIP score with the documented formula (`discount^ticks * size`, walked to `target_size`, normalized per side) and accrues the expected share of the pool.
`pmus/ledger.py`	Append-only SQLite (`pmus/data/ledger.db`) for quotes, orders, fills, rebates, incentive snapshots, shadow fills and LIP estimates. Both modes write here.
`pmus/report.py`	`python -m pmus report --since 24h`. Shows shadow or live P&L split into spread capture, rebates, estimated LIP, fees and inventory mark-to-market.
`pmus/__main__.py`	The CLI: `run --mode shadow|live`, `report`, `probe` (one-shot dump of live markets and incentives), `flatten` (cancel all and

File	Responsibility
	close positions).
`pmus/logging_setup.py`	A rotating file log in `pmus/logs/` plus the console.
`pmus/tests/`	Pytest with recorded JSON fixtures from the public gateway. No network in unit tests.
`deploy/pmus.service`, `deploy/Dockerfile`, `pmus/README.md`	systemd unit with `Restart=always` for an Ubuntu EC2 host, and run docs.

Phases

Each phase ends in a check, and is verified before the next starts.

1. **Scaffold, models, fees, venue reads.**
2. Verify: `pytest pmus/tests -q` passes.
3. Verify: `python -m pmus probe` prints live markets, incentive windows and one book from the public gateway.
4. **Pure strategies.**
5. Verify: table-driven tests for `desired_quotes` (join best, skip on max_spread, inventory skew, size from budget, tick rounding).
6. Verify: tests for `scan` (fee math against the documented examples, and both directions).
7. **Shadow engine, ledger, report.**
8. Verify: `python -m pmus run --mode shadow` runs 30 minutes against live data with no exceptions.
9. Verify: `report` shows non-empty estimates.
10. Verify: the log file rotates.
11. **Live execution path.** Covers the SDK auth wrapper, post-only batched create and cancel, private websocket order and position sync, and risk gates.
12. Verify: tests with a fake venue confirm that the reconcile diff, gate breaches, the maintenance pause and the kill switch all behave correctly.
13. Verify: no live order is placed in this phase.
14. **Ops.**
15. Verify: the systemd unit and Dockerfile build.
16. Verify: README covers env vars, shadow and live start, `flatten`, and log locations.

Go-live gate (your call). - Run 72 hours of shadow on the AWS box. If `report` shows a positive net result (estimated LIP + rebates + spread − fees − mark-to-market), start live with caps at $25 of total collateral. - If the shadow result is flat or negative, we change the strategy or stop.

Throughput checkpoint

- **Blocking first steps.** `models.py` and `venue.py` come first. Everything else imports them.
- **Independent workstreams.** After phase 1, `lip_quoter`, `basket_scanner` and `fees` are separate files with no shared writes. They're small, so one owner does them in sequence.
- **Shared mutable state.** None across workers. At runtime the engine is the only writer of order state. `ledger.db` has one writer process.

- **Smallest safe decomposition.** One code owner (Flash 3.8 High, as you asked), because the phases share types and each depends on the last. I review each phase's diff and run the checks myself before the next phase starts.

`architect` skipped: this is a new package with a single delegate, and the design is the data shapes and module table above. If the reconcile engine shape is contested in review, I'll run `interrogate` on it before live.

Risks and open items

- **Florida eligibility** isn't confirmed. You'll see it during sign-up.
- **The shadow fill model is an estimate.** Queue position is approximated, so live fills will be worse.
- **Reward payout floor.** It's $1.00, and the docs don't say whether it applies per window or per payout. Live earnings could be zero even with a positive score.
- **Server region.** Any region works for Polymarket US. Ireland adds about 80 ms each way to US servers. A US region like us-east-1 is cheaper on latency.